MODULE 11 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

Amazon API Gateway

Three API types, four integration styles, five ways to authorise - and the one step everyone forgets.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 11 of 34

admin@sthithapragna.com

API Gateway is a **configurable proxy**. Every question is really asking which piece of configuration handles the requirement.

The three API types

TYPE	WHAT IT IS FOR
REST API	The full-featured one. API keys, usage plans, caching, request validation, WAF
HTTP API	Cheaper and faster, roughly 71% less per request. Fewer features
WebSocket API	Persistent two-way connections. The server can push to the client

ON THE EXAM

Confusingly, the REST API is the older and more capable one. HTTP APIs are newer and deliberately simpler, not more advanced.

REST API or HTTP API?

REST API

When you need the features

- API keys and usage plans
- Response caching
- Request and response validation
- Mapping templates and transformations
- WAF attaches directly

HTTP API

When you need cheap and fast

- About 71% cheaper per request
- Lower latency
- Native JWT authoriser
- Simple proxying to Lambda or HTTP
- No caching, no usage plans, no WAF

THE TIEBREAKER

Any mention of API keys, tiered quotas, caching, request validation or WAF selects the REST API. Otherwise HTTP APIs are the cheaper default.

The request pipeline

01

Method request

Authorisation, API keys, request validation.

›

02

Integration request

Transform and forward to the backend.

›

03

Integration response

Transform what the backend returned.

›

04

Method response

Status codes and headers returned to the caller.

ON THE EXAM

Four stages, and questions locate a requirement in one of them. Validation belongs to the method request; transformation to the integration stages.

Integration types

- **Lambda proxy** passes the whole request through and expects a shaped response back.
- **Lambda custom** uses mapping templates to build the request and the response.
- **HTTP** and **HTTP proxy** do the same for any HTTP backend.
- **AWS service** integration calls another AWS API directly, with no Lambda in between.
- **Mock** returns a canned response from the gateway itself. Useful for stubs and CORS.

> ON THE EXAM
>
> The proxy integrations are the modern default. The custom ones exist for when the payload must be reshaped without touching the backend.

The Lambda proxy contract

- The event carries **path, headers, queryStringParameters, pathParameters and body**.
- The body arrives as a **string**, not a parsed object. You parse it yourself.
- The response must contain **statusCode**, and **body** must also be a string.
- Optional: **headers**, **multiValueHeaders** and **isBase64Encoded** for binary payloads.
- Return an object instead of a string body and API Gateway answers **502 Bad Gateway**.

> ON THE EXAM
>
> A 502 from a Lambda-backed API almost always means the function returned the wrong shape. The function succeeded; the gateway could not use the result.

Proxy or custom integration?

PROXY INTEGRATION

The default

- Whole request passed straight through
- No mapping templates to maintain
- Backend controls status and headers
- Add a route without touching the gateway
- Backend must know the event shape

CUSTOM INTEGRATION

When the shape must change

- Mapping templates reshape both ways
- Backend stays unaware of API Gateway
- Suits legacy or third-party backends
- VTL is fiddly and hard to test
- Every change means a gateway change

THE TIEBREAKER

Custom integration earns its place when you cannot change the backend. If you own the backend, proxy is simpler and easier to evolve.

Mapping templates

- Written in **Velocity Template Language**, applied on the integration request or response.
- Reshape JSON, rename fields, inject values, or convert XML to JSON.
- Access request data through **$input** and context through **$context**.
- **$context.authorizer** carries values an authoriser returned, so the backend can use them.
- REST APIs only. HTTP APIs support only simple parameter mappings.

> ON THE EXAM
>
> The usual exam use: a client sends one format, the backend expects another, and neither can change. That is a mapping template.

Request validation

- Validate required **query string parameters and headers**, and the **body against a JSON Schema model**.
- Invalid requests are rejected with **400** at the gateway. The backend is never invoked.
- That saves Lambda invocations, and removes validation code from every function.
- Models are defined per API and reused across methods.
- REST APIs only. It is a common reason to choose REST over HTTP.

> ON THE EXAM
>
> When a scenario wants malformed requests stopped without invoking the backend, and without writing validation code, this is the answer.

Skipping the backend entirely

- **Mock integration** returns a response the gateway generates. No backend is called.
- Used for stub endpoints during development, and for answering CORS preflight.
- **AWS service integration** calls another AWS API directly from the gateway.
- API Gateway straight to SQS, SNS, Step Functions, Kinesis or DynamoDB, with no Lambda.
- Removes a function that existed only to forward a request. Less cost, less latency, less code.

> ON THE EXAM
>
> When a scenario describes a Lambda function that only writes the request to a queue, a direct service integration is very often the intended answer.

OpenAPI import and export

- An entire API can be defined in an **OpenAPI** document and imported.
- AWS-specific behaviour lives in **x-amazon-apigateway-** extensions.
- Export a working API to OpenAPI to hand a contract to client teams.
- SAM and CloudFormation can define an API this way rather than resource by resource.
- Importing replaces the definition. It still does not deploy it to any stage.

> ON THE EXAM
>
> Note the last line. Even an import leaves the stage serving the old deployment until you create a new one.

Stages and deployments

- **Configuration changes do nothing until you deploy.** This catches almost everyone once.
- A **deployment** is a snapshot of the API. A **stage** points at one deployment.
- Each stage has its own URL, its own settings and its own throttling and caching.
- Typical stages: dev, test and prod, all from the same API definition.
- Rolling back is repointing the stage at an earlier deployment. It is instant.

> ON THE EXAM
>
> 'I changed the configuration and nothing happened' is a deployment question, not a bug. It is a favourite exam scenario precisely because it is so easy to do.

Stage variables

- Name and value pairs attached to a stage, referenced as **${stageVariables.name}**.
- Point each stage at a different Lambda **alias**, so dev calls dev and prod calls prod.
- Or at a different HTTP endpoint, or a different HTTP connection ID.
- Available in mapping templates and passed to Lambda in the context object.
- They are configuration, not secrets. They are visible to anyone who can read the stage.

> ON THE EXAM
>
> The classic pairing is stage variables with Lambda aliases: one API definition, one function, and the stage decides which version runs.

Canary release deployments

- A stage can send a **percentage of traffic** to a canary deployment and the rest to production.
- The canary can have its own stage variables, so it can call a different backend version.
- CloudWatch metrics are reported **separately** for the canary.
- Promote when it looks healthy, or delete the canary to roll back instantly.
- REST APIs only. It also does not shift traffic between Lambda versions - aliases do that.

> ON THE EXAM
>
> Two independent traffic-shifting mechanisms exist: canary deployments at the gateway, and weighted aliases at Lambda. Module 24 covers the second.

Five ways to authorise

MECHANISM	USE IT WHEN
IAM authorisation	Callers are AWS principals signing with SigV4
Cognito user pool authoriser	End users sign in and present a Cognito JWT
Lambda authoriser	Any custom logic: a third-party IdP, or per-tenant rules
JWT authoriser	HTTP APIs only. Any OIDC provider, natively
Resource policy	Restrict by source VPC, IP range or AWS account

ON THE EXAM

API keys are not on this list. They identify a caller for metering, and are explicitly not an authentication mechanism.

Lambda authorisers

- **TOKEN** type receives a single header, typically the bearer token.
- **REQUEST** type receives headers, query strings, paths and context. Far more flexible.
- It returns an **IAM policy** allowing or denying the method, plus an optional context object.
- **Results are cached**, keyed by the token or the identity sources, for up to 3600 seconds.
- Caching is why a revoked token can keep working. Reduce the TTL if that matters.

> ON THE EXAM
>
> The returned context reaches the backend as $context.authorizer, which is how a tenant ID or user role gets passed on without a second lookup.

Cognito user pool authorisers

- The gateway validates a Cognito **ID token** or access token itself. No Lambda involved.
- Signature, expiry and issuer are all checked for you.
- Token claims reach the backend through **$context.authorizer.claims**.
- It authenticates only. It does not do fine-grained per-resource authorisation.
- For rules beyond 'is this a valid user', use a Lambda authoriser instead.

> ON THE EXAM
>
> The dividing line is simple. Cognito authoriser answers 'is this a valid signed-in user'. Anything more nuanced needs a Lambda authoriser.

API keys and usage plans

- **An API key is not authentication.** It identifies a caller so you can meter and throttle them.
- A **usage plan** ties API keys to stages, with a **rate**, a **burst** and a **quota**.
- Rate and burst are per second. Quota is per day, week or month.
- This is how tiered access is sold: bronze, silver and gold plans on one API.
- REST APIs only. HTTP APIs have no API keys and no usage plans.

> ON THE EXAM
>
> Any scenario about per-customer quotas or selling tiers of API access is a usage plan question, and therefore a REST API question.

Throttling, layer by layer

- **Account level:** 10,000 requests per second steady, with a 5,000 request burst bucket, per Region.
- **Stage and method level:** limits you set, applying to everyone.
- **Usage plan level:** per API key, so different customers get different limits.
- The **most restrictive** applicable limit wins, and exceeding any of them returns **429**.
- The account burst quota is **fixed** and cannot be increased. The steady rate can.

> ON THE EXAM
>
> Throttling here protects your backend as well as the platform. A 429 at the gateway is far cheaper than a database falling over behind it.

Response caching

- Enabled **per stage**, sized from **0.5 GB to 237 GB**, and billed hourly while it exists.
- Default TTL is **300 seconds**. Setting it to 0 disables caching for that method.
- The cache key is built from method request parameters you nominate.
- Clients can bypass it with **Cache-Control: max-age=0**, if you grant that permission.
- REST APIs only. Cache invalidation requires an IAM permission, so it can be restricted.

> ON THE EXAM
>
> Caching reduces backend load and cost, and it is billed hourly whether or not it gets hits. It is not free the way a usage plan is.

CORS at the gateway

- The browser sends an **OPTIONS** preflight, which your API must answer.
- The gateway can answer it with a **mock integration**, generated by Enable CORS.
- With **proxy integration**, your backend must also return the CORS headers on the real response.
- Enabling CORS in the console handles OPTIONS only. That is the usual half-fix.
- HTTP APIs configure CORS declaratively and handle both sides for you.

> ON THE EXAM
>
> The classic symptom: preflight succeeds, the real request is blocked. The OPTIONS method is configured; the proxy backend is not returning the headers.

Endpoint types and custom domains

- **Edge-optimised:** requests enter at a CloudFront edge location. The default for REST APIs.
- **Regional:** served from the Region. Better when callers are nearby, or you want your own CloudFront.
- **Private:** reachable only from inside a VPC, through an interface VPC endpoint.
- **Custom domain names** replace the execute-api URL, and need an ACM certificate.
- For an edge-optimised custom domain, the certificate must be in **us-east-1**.

> ON THE EXAM
>
> That last line is a classic gotcha. Edge-optimised means CloudFront, and CloudFront only reads certificates from us-east-1.

Reading the error codes

CODE	WHAT IT MEANS HERE
400	Bad request. Often request validation rejecting the payload
403	Access denied, WAF block, or a resource policy refusing the caller
429	Throttled. Account, stage, method or usage plan limit exceeded
502	Backend returned something unusable. Usually a malformed Lambda proxy response
504	Integration timed out. The backend took longer than the integration timeout

ON THE EXAM

502 against 504 is the pairing that matters. 502 means the backend answered wrongly; 504 means it did not answer in time.

Currency note: the 29-second limit moved twice

- **The examinable answer:** the integration timeout is **29 seconds**, and 504 is what you get past it.
- **June 2024:** raisable via Service Quotas up to **300 seconds**, on Regional and private REST APIs.
- That increase may require **reducing your account throttle quota**. It is a trade, not a free upgrade.
- **November 2025:** response streaming on REST APIs extends effective timeouts to 15 minutes.
- Edge-optimised REST APIs and HTTP APIs are eligible for **neither** increase.

> ON THE EXAM
>
> Two separate changes to the same number, both after exam guide v2.1. Answer 29 seconds, and know that a real system has two ways past it.

Observability

- **Execution logs** record what the gateway did internally. Verbose, and off by default.
- **Access logs** record one line per request, in a format you define with $context variables.
- **X-Ray** traces the request through the gateway and into the backend.
- Key metrics: **Count**, **4XXError**, **5XXError**, **Latency** and **IntegrationLatency**.
- Latency minus IntegrationLatency is the gateway's own overhead. Usually a few milliseconds.

> ON THE EXAM
>
> That last line is a real diagnostic. If Latency is high but IntegrationLatency is low, the delay is in the gateway - often an authoriser.

Two of these hinge on a feature existing only on REST APIs. Check the API type before you check anything else.

A collaborative document editor must push changes to every connected client the moment another user makes an edit. Clients currently poll a REST endpoint every two seconds, which is expensive and still feels laggy.

Sessions last hours, several hundred clients may be connected to one document, and the server must be able to initiate messages rather than only respond to them.

What should the team build?

A. A REST API with response caching and a shorter polling interval

B. A REST API with response streaming to each client

C. An API Gateway WebSocket API, storing connection IDs in DynamoDB

D. An HTTP API with a JWT authoriser and server-sent events

◆ PAUSE AND DECIDE

What should the team build?

A Cache plus faster polling

Still polling. More frequent polls means more cost, and caching makes the data staler rather than fresher.

B REST with response streaming

Streaming sends one response progressively to one caller. It cannot push an unrelated later event, and it cannot fan out to other clients.

C WebSocket API with connection IDs

A persistent bidirectional connection is exactly the requirement. Storing connection IDs lets the backend push to every client on a document.

D HTTP API with SSE

Server-sent events are one-directional and API Gateway HTTP APIs do not support streaming, so this cannot be built as described.

THE SIGNAL

'The server must be able to initiate messages' is the deciding clause. Only a WebSocket API gives the backend a way to reach a client unprompted.

A developer updates a REST API to add request validation on the POST method and changes the integration to point at a new Lambda alias. They save both changes in the console and confirm the settings show correctly when they reopen the method.

Calls to the production URL still reach the old alias and still accept invalid payloads. Nothing appears in CloudWatch to suggest an error.

What is the most likely explanation?

A The changes were saved but never deployed to the stage, so the stage still serves the previous deployment

B The API Gateway cache is serving stale responses and needs invalidating

C Request validation only applies to GET methods

D The Lambda alias needs a resource-based policy before API Gateway can invoke it

◆ PAUSE AND DECIDE

What is the most likely explanation?

A Never deployed

Configuration is saved against the API, and a stage serves a deployment snapshot. Until a new deployment is created, the stage keeps serving the old one.

B Stale cache

Caching would affect responses, not which alias is invoked, and it would not make invalid payloads pass validation.

C Validation is GET-only

Simply false. Body validation is most useful on POST and PUT, which are the methods that carry bodies.

D Missing resource policy

That would produce visible 500 errors and permission failures in CloudWatch. The scenario says calls succeed against the old alias.

THE SIGNAL

'Settings look correct but behaviour has not changed, and no errors anywhere' is the fingerprint. Nothing is broken - the change simply is not live.

A company sells access to a weather data API. It needs three tiers: free at 10 requests per second and 10,000 per month, standard at 100 per second and 1 million per month, and enterprise at 1,000 per second with no monthly cap.

Excess requests must be rejected at the gateway without invoking the backend, and each customer's usage must be reportable. Customers already authenticate with Cognito.

How should this be implemented?

A An HTTP API with a JWT authoriser and per-route throttling

B A Lambda authoriser that counts requests per customer in DynamoDB and denies over-limit calls

C AWS WAF rate-based rules, one rule per customer tier

D A REST API with usage plans and API keys, one plan per tier, alongside the Cognito authoriser

◆ PAUSE AND DECIDE

How should this be implemented?

A HTTP API with per-route throttling

HTTP APIs have no API keys and no usage plans, so per-customer quotas are impossible. Route throttling applies to everyone equally.

B Lambda authoriser counting in DynamoDB

Rebuilds usage plans by hand, invokes a function on every request including rejected ones, and adds a DynamoDB write to the request path.

C WAF rate-based rules

WAF rate rules work on IP addresses over five-minute windows. They cannot express per-customer monthly quotas.

D REST API with usage plans and API keys

Usage plans express rate, burst and quota per key exactly as described, reject at the gateway, and produce per-key usage reports.

THE SIGNAL

The Cognito detail is a distractor. Authentication and metering are separate jobs, and this API needs both: Cognito for identity, API keys for the tier.

> A Node.js Lambda function behind a REST API with proxy integration returns 502 Bad Gateway for every request. CloudWatch shows the function completing successfully in about 40 ms, with no errors logged and no timeouts.
>
> The handler ends by returning an object containing a status field and a data object.

What is the cause?

A The function timeout is too low for API Gateway's integration timeout

B The response does not match the proxy integration format: it needs statusCode, and body must be a string

C API Gateway lacks permission to invoke the function

D The API has not been deployed since the integration was configured

◆ PAUSE AND DECIDE

What is the cause?

A Function timeout too low

A timeout produces 504, not 502, and the function completes in 40 ms. Both facts rule this out.

B Wrong proxy response shape

Proxy integration requires statusCode and a string body. Returning a bare object means the gateway cannot build a response, so it answers 502.

C Missing invoke permission

That produces 500 with a clear permission error, and the function would never run. Here it runs successfully.

D Not deployed

Scenario 2's answer, and wrong here. An undeployed change means old behaviour, not a 502 on every request.

THE SIGNAL

'The function succeeded and the API still failed' is the whole diagnosis. The failure is between Lambda and the gateway, which is what 502 means.

What to carry forward

- **Six features are REST-only:** API keys, usage plans, caching, request validation, mapping templates, WAF.
- **Nothing takes effect until you deploy to a stage.** The most common mistake there is.
- **502** means a bad backend response. **504** means the backend was too slow.
- API keys meter, they do not authenticate. Pair them with a real authoriser.
- Direct AWS service integrations remove Lambda functions that only forward requests.

> ON THE EXAM
>
> Next: decoupling with SQS, SNS and EventBridge - and the direct integrations from this module become far more useful once those exist.